

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA12	Course Title:	Computer Architecture for Multicore Systems
CO Assessed:	CO4	Assessment:	ASSESSMENT TOOL1- INDUSTRY PROBLEM SOLVING TASK
Weightage:	30%	Total Marks:	25
CO4 Statement:	<i>Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.</i>		
Student Name:		Reg.No.:	
Department:		Date:	

ANNEXURE A
INDUSTRY PROBLEM SOLVING TASK

- Smartphone Performance Optimization**
A mobile manufacturer observes lag while switching between apps. Analyze the role of L1, L2, L3 cache and DRAM and propose a hierarchical memory redesign to reduce latency and improve throughput.
- Cloud Server Memory Bottleneck**
A cloud service provider experiences high latency in database queries. Compare cache hierarchy vs virtual memory paging and recommend improvements using appropriate memory technologies.
- Autonomous Vehicle Data Processing**
An autonomous vehicle must process sensor data in real time. Evaluate SRAM, DRAM, and MRAM and design a memory hierarchy that maximizes bandwidth and minimizes latency.
- AI Inference Edge Device**
An edge AI device needs fast model loading with low power consumption. Compare NAND Flash, DRAM, and RRAM and recommend an optimized hierarchical memory structure.
- High-Performance Gaming Console**
A gaming console suffers frame drops during large scene loading. Analyze L3 cache vs DRAM throughput and propose memory hierarchy enhancements.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Understanding of Industry Problem (20%)	Application of Engineering Concepts (25%)	Solution Design (25%)	Use of Tools (15%)	Analysis (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

1. Smartphone Performance Optimization

When switching between apps, the smartphone needs to quickly access instructions and data from memory. If the required data is not available in the faster cache, the processor has to access DRAM, which takes more time. This can cause lag while switching between applications.

Role of Different Memory Levels

- **L1 Cache:** It is the fastest and smallest cache. It stores frequently used instructions and data needed by the CPU. This helps reduce access latency.
- **L2 Cache:** It is larger than L1 but slightly slower. It stores data that cannot fit in L1 and reduces the need to access DRAM.
- **L3 Cache:** It is larger than L2 and is usually shared between CPU cores. It acts as an additional buffer before accessing DRAM.
- **DRAM:** It is the main memory where active applications and their data are stored. It has much higher latency than cache but provides larger capacity.

Proposed Hierarchical Memory Redesign

To reduce app-switching lag, the smartphone can use the following hierarchy:

CPU → L1 Cache → L2 Cache → L3 Cache → DRAM

The redesign can include:

1. **Increase L1 cache efficiency** by keeping frequently accessed instructions and data closer to the CPU.
2. **Optimize L2 cache size** so that more application data can be stored without increasing latency significantly.
3. **Use a larger shared L3 cache** to improve data sharing between CPU cores and reduce DRAM accesses.
4. **Use high-speed DRAM** with higher bandwidth to transfer application data faster.
5. **Improve cache replacement and prefetching** so that frequently required data is loaded into the cache before the CPU requests it.

Expected Improvement

The redesigned hierarchy reduces the number of accesses to DRAM, which decreases **memory access latency**. A larger and more efficient cache also improves the **cache hit ratio**, allowing the processor to obtain data faster. High-bandwidth DRAM further improves the transfer of large amounts of application data.

Therefore, an optimized **L1–L2–L3 cache and high-speed DRAM hierarchy** can reduce app-switching latency, improve throughput and provide smoother smartphone performance.

2. Cloud Server Memory Bottleneck

A cloud service provider is experiencing high latency in database queries because the processor may need to access main memory or storage frequently. Proper use of cache hierarchy and virtual memory can reduce this problem.

Cache Hierarchy vs Virtual Memory Paging

Cache hierarchy consists of L1, L2 and L3 caches. L1 cache is the fastest and smallest, followed by L2 and L3. Frequently accessed database data and instructions can be stored in cache so that the CPU can access them quickly. This reduces memory access latency and improves throughput.

Virtual memory paging uses secondary storage when the available DRAM is insufficient. The data is divided into pages, and required pages are loaded into RAM. If a required page is not present in RAM, a page fault occurs. Since storage access is much slower than DRAM, frequent page faults can significantly increase database query latency.

Feature	Cache Hierarchy	Virtual Memory Paging
Main purpose	Fast data access	Extend available memory
Speed	Very high	Lower
Latency	Very low	High during page faults
Storage	Small	Large
Effect on queries	Improves response time	Can increase response time if paging is frequent

Recommended Improvements

1. **Increase L3 cache capacity** to store more frequently accessed database data and reduce DRAM accesses.
2. **Optimize L1 and L2 cache usage** so frequently used instructions and data are available closer to the CPU.
3. **Increase DRAM capacity** to keep more database working sets in physical memory and reduce page faults.
4. Use **high-bandwidth DRAM** to improve data transfer between the processor and main memory.
5. Reduce unnecessary paging by allocating sufficient RAM for frequently accessed databases.
6. Use **NAND Flash-based SSDs** instead of slower storage for virtual memory and database storage when storage access is unavoidable.

Conclusion

Cache hierarchy and virtual memory solve different problems. L1, L2 and L3 caches reduce CPU memory access latency, while virtual memory provides additional memory capacity through paging. For a cloud database server, the best approach is to use an efficient cache hierarchy, sufficient high-bandwidth DRAM and fast NAND Flash storage. Reducing page faults and increasing cache hits will lower database query latency and improve overall system throughput.

3. Autonomous Vehicle Data Processing

An autonomous vehicle continuously collects data from cameras, LiDAR, radar and other sensors. This data must be processed in real time for tasks such as object detection, lane detection and decision making. Therefore, the memory system should provide low latency and high bandwidth.

Evaluation of SRAM, DRAM and MRAM

- SRAM: It has very low latency and high speed, making it suitable for storing frequently accessed sensor data and processor instructions. However, it is expensive and has lower storage capacity.
- DRAM: It provides much larger capacity than SRAM and good bandwidth at a lower cost. It can store large amounts of sensor data and intermediate processing results, but its latency is higher than SRAM.
- MRAM: It is a non-volatile memory with fast access and low power consumption. It can retain data even when power is removed and can be used for important data that needs fast access.

Proposed Memory Hierarchy

The memory hierarchy for an autonomous vehicle can be designed as:

CPU/AI Processor → SRAM → DRAM → MRAM

1. SRAM is placed closest to the processor to store frequently accessed sensor data, instructions and temporary processing results.
2. DRAM is used as the main working memory for large sensor datasets and AI processing.
3. MRAM can be used for storing important parameters, frequently required data and system information because it is fast and non-volatile.
4. Efficient data prefetching and caching can be used to bring required sensor data into SRAM before processing.

Comparison

Memory	Latency	Bandwidth	Capacity	Suitable Use
SRAM	Very Low	Very High	Low	CPU/AI cache
DRAM	Low	High	High	Main memory
MRAM	Low	High	Moderate	Fast non-volatile storage

Conclusion

For an autonomous vehicle, SRAM should be used for the most time-critical data, DRAM should handle large sensor datasets, and MRAM can provide fast non-volatile storage for important system data. This hierarchy keeps frequently required data close to the processor, reducing memory access latency while maintaining high bandwidth. Thus, the proposed design can support real-time sensor processing and improve the overall performance of the autonomous vehicle.

4. AI Inference Edge Device

An edge AI device needs to load AI models quickly and process them with low power consumption. The memory system should provide enough storage capacity while reducing access time and power usage. NAND Flash, DRAM and RRAM can be arranged in a hierarchy to achieve this.

Comparison of NAND Flash, DRAM and RRAM

- **NAND Flash:** It is non-volatile and provides high storage capacity. It can store large AI models even when the device is powered off. However, its access latency is higher than DRAM and RRAM.
- **DRAM:** It provides faster access and high bandwidth compared to NAND Flash. It can be used to load the AI model and store intermediate data during inference. However, it is volatile and consumes power for refreshing.
- **RRAM:** It is non-volatile and can provide low access latency with low power consumption. It is suitable for storing frequently used AI model parameters and can reduce the need for repeated data movement.

Proposed Hierarchical Memory Structure

The optimized memory hierarchy can be:

AI Processor → RRAM → DRAM → NAND Flash

1. **RRAM** can be placed close to the AI processor to store frequently used model parameters and critical data. This provides fast access with lower power consumption.
2. **DRAM** can be used as the main working memory for loading the complete AI model and storing intermediate results during inference.
3. **NAND Flash** can be used for permanent storage of large AI models, applications and other data.
4. Frequently used models can be transferred from NAND Flash to DRAM before execution, while important parameters can be kept in RRAM for faster access.

Comparison Table

Memory	Latency	Power Consumption	Capacity	Main Use
RRAM	Low	Low	Moderate	Frequently used model data
DRAM	Low	Moderate	High	Model execution
NAND Flash	High	Low	Very High	Permanent model storage

Conclusion

The proposed RRAM–DRAM–NAND Flash hierarchy provides a good balance between speed, capacity and power consumption. NAND Flash stores large AI models permanently, DRAM provides high-speed working memory, and RRAM provides fast and low-power access to frequently used model data. This reduces model loading time, data movement and power consumption, making the edge AI device more efficient for real-time inference.

5. High-Performance Gaming Console

A gaming console may experience frame drops when loading large game scenes because a large amount of texture, animation, geometry and game data must be transferred between memory and the processor. If the required data is not available in cache, the system has to access DRAM, which can increase latency and reduce performance.

L3 Cache vs DRAM Throughput

- **L3 Cache:** L3 cache is faster than DRAM and has lower latency. It stores frequently accessed game data and instructions close to the CPU. However, its capacity is limited, so it cannot hold the complete data required for large game scenes.
- **DRAM:** DRAM has much higher capacity than L3 cache and is suitable for storing large textures, game assets and other scene data. However, its latency is higher, so frequent DRAM accesses can affect frame rendering.

Feature	L3 Cache	DRAM
Latency	Very Low	Higher
Capacity	Small	Large
Throughput	High	High
Cost per bit	High	Lower
Main Use	Frequently used data	Large game data

Proposed Memory Hierarchy Enhancement

The console can use the following hierarchy: CPU/GPU → L1/L2 Cache → Large L3 Cache → High-Bandwidth DRAM → SSD

1. **Increase L3 cache capacity** so that more frequently accessed game data can remain close to the processor.
2. Use **high-bandwidth DRAM** to transfer large textures and scene data quickly.
3. Improve **cache prefetching** so that game data required for upcoming frames is loaded into the cache in advance.
4. Use **memory compression** for textures and other data to reduce the amount of data transferred from DRAM.
5. Use a **fast SSD** for storing large game assets and stream the required data into DRAM before it is needed.
6. Give the GPU sufficient memory bandwidth because modern games require large amounts of data for graphics processing.

Conclusion

L3 cache provides low-latency access, while DRAM provides the large capacity and high bandwidth needed for large game scenes. Increasing L3 cache efficiency, using high-bandwidth DRAM, improving prefetching and faster data streaming from SSD can reduce memory delays. This helps maintain a steady flow of data to the CPU and GPU and reduces frame drops during large scene loading.